

SIMPLON

Formation Data Engineer

Industrial IoT Data Lake

**Conception et mise en œuvre d'un Data Lake industriel avec Docker,
MinIO, Apache Airflow et OpenMetadata**

Rapport de projet

Réalisé par:
Araïlym PERNEBAY

Encadrant:
Bertrand FOURNEL

Juillet 2026

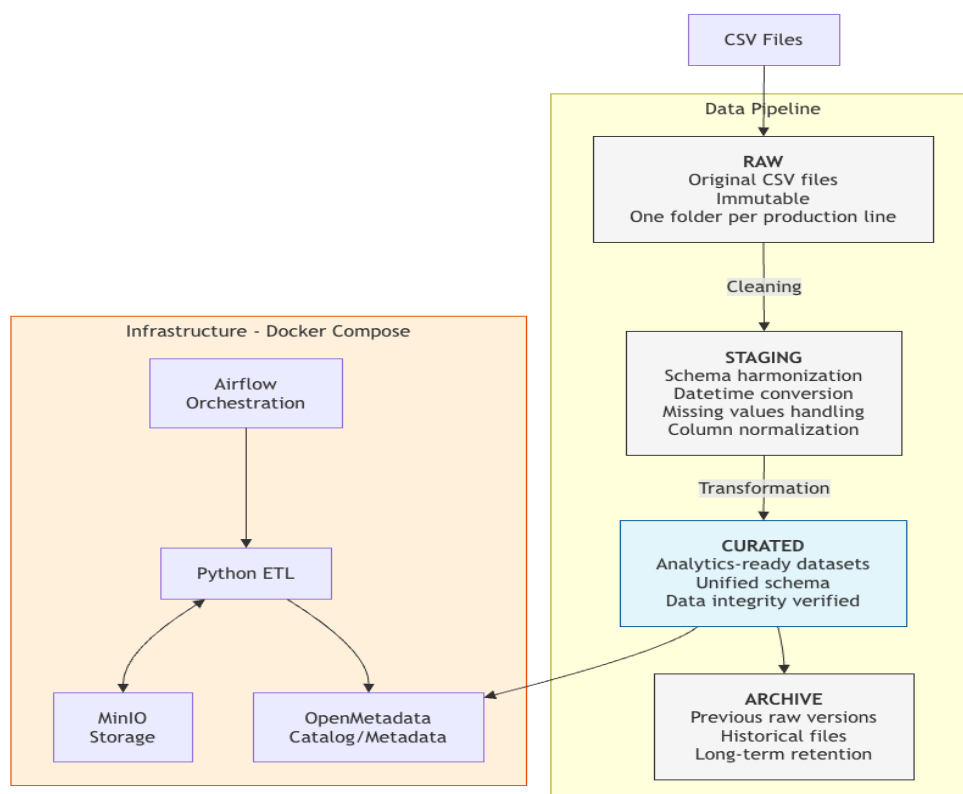
1. Introduction

Avec le développement de l'Industrie 4.0, les systèmes industriels produisent quotidiennement d'importants volumes de données provenant de capteurs, de lignes de production et d'équipements de surveillance. La collecte, le stockage et l'exploitation de ces données sont devenus des enjeux majeurs pour les entreprises souhaitant améliorer leurs processus de production et prendre des décisions basées sur les données.

Les bases de données relationnelles traditionnelles présentent certaines limites lorsqu'il s'agit de gérer de grands volumes de données hétérogènes. C'est pourquoi de nombreuses organisations adoptent aujourd'hui une architecture de type **Data Lake**, qui permet de stocker des données structurées, semi-structurées et non structurées dans leur format d'origine. Cette approche offre une grande flexibilité pour les traitements futurs, l'analyse des données et les applications d'intelligence artificielle.

L'objectif de ce projet est de concevoir et de mettre en œuvre un **Data Lake industriel** capable de centraliser les données issues de plusieurs lignes de production tout en automatisant leur ingestion, leur transformation et leur catalogage. L'architecture repose sur une organisation en plusieurs couches de stockage (**Raw**, **Staging**, **Curated** et **Archive**) afin de garantir une gestion claire du cycle de vie des données.

Figure 1 : Architecture générale du projet



Afin de construire cette plateforme, plusieurs technologies open source ont été utilisées :

- **MinIO** a été choisi comme solution de stockage objet compatible avec le protocole Amazon S3. Il constitue le cœur du Data Lake en hébergeant les différentes couches de données.

- **Apache Airflow** assure l'orchestration des traitements grâce à des workflows (DAGs) permettant d'automatiser les différentes étapes du pipeline de données.
- **Python** est utilisé pour développer les traitements ETL (Extract, Transform, Load), notamment le nettoyage des données, l'harmonisation des schémas et la vérification de leur intégrité.
- **OpenMetadata** permet de cataloguer les ressources stockées dans le Data Lake afin de centraliser les métadonnées et de faciliter la découverte des données.
- Enfin, **Docker Compose** simplifie le déploiement de l'ensemble des services en exécutant chaque composant dans un conteneur indépendant tout en assurant leur communication au sein d'un même environnement.

Le résultat obtenu est un prototype fonctionnel illustrant les principales étapes d'une chaîne de traitement de données moderne : depuis l'ingestion de fichiers CSV industriels jusqu'à leur organisation dans un Data Lake, leur transformation automatisée et leur catalogage dans OpenMetadata. Cette architecture constitue une base solide pouvant être enrichie ultérieurement par des traitements analytiques avancés ou des applications de maintenance prédictive.

2. Architecture et technologies utilisées

Afin de réaliser ce projet, une architecture composée de plusieurs outils open source a été mise en place. Chaque composant remplit un rôle précis dans la chaîne de traitement des données, depuis l'ingestion des fichiers CSV jusqu'à leur catalogage.

L'ensemble de la plateforme a été déployé localement à l'aide de **Docker Compose**, permettant d'exécuter chaque service dans un conteneur indépendant. Cette approche facilite la reproductibilité du projet et simplifie les échanges entre les différents composants.

Le stockage des données est assuré par **MinIO**, une solution de stockage objet compatible avec l'API Amazon S3. Dans ce projet, MinIO constitue le Data Lake et contient quatre buckets correspondant aux différentes étapes du traitement des données :

- **Raw** : stockage des fichiers CSV d'origine provenant des lignes de production. Les données sont conservées dans leur format initial afin de préserver une copie fidèle des informations collectées.
- **Staging** : stockage des données après les premières opérations de transformation. Les schémas des différents fichiers sont harmonisés afin d'obtenir une structure commune.
- **Curated** : stockage des jeux de données nettoyés et standardisés, prêts à être exploités pour des analyses ultérieures.
- **Archive** : conservation des anciennes versions des fichiers afin de maintenir un historique des données traitées.

L'automatisation du traitement est réalisée avec **Apache Airflow**. Un DAG (Directed Acyclic Graph) a été développé afin d'orchestrer les différentes étapes du pipeline. Ce workflow prend en charge l'ingestion des fichiers CSV dans MinIO ainsi que les opérations de transformation nécessaires au passage des données entre les différentes couches du Data Lake.

Les traitements ETL ont été développés en **Python** (*transform_staging_dag.py*). Les scripts réalisés permettent notamment de :

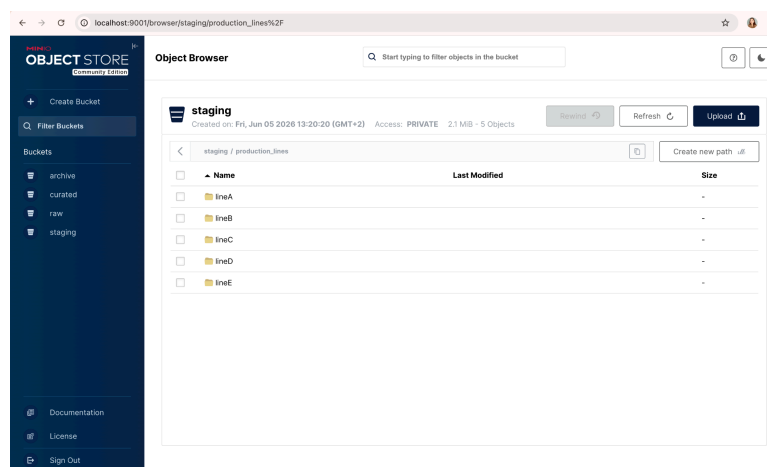
- lire les fichiers CSV provenant des différentes lignes de production ;
- harmoniser les noms des colonnes lorsque les schémas diffèrent ;

- nettoyer les données avant leur stockage ;
- vérifier l'intégrité des fichiers à l'aide d'un calcul de hachage MD5 (*verify_integrity_hash.py*) ;
- enregistrer les résultats dans les buckets appropriés.

Enfin, **OpenMetadata** a été intégré afin de cataloguer le Data Lake. Le service de stockage MinIO a été enregistré dans OpenMetadata, permettant la découverte automatique des buckets (*Raw*, *Staging*, *Curated* et *Archive*). Un workflow d'ingestion des métadonnées a également été exécuté avec succès via le service d'ingestion d'OpenMetadata reposant sur Airflow (*task_ingestion_pipeline*). Cette étape permet de centraliser les informations décrivant le stockage des données et facilite leur consultation au sein du catalogue.

L'architecture obtenue sépare clairement les responsabilités entre le stockage (MinIO), l'orchestration (Airflow), le traitement des données (Python) et la gestion des métadonnées (OpenMetadata). Cette organisation rend le pipeline plus lisible, plus facilement maintenable et permet d'isoler chaque étape du traitement des données.

Figure 2 : Exemple d'architecture de la couche STA détaillée du Data Lake



3. Implémentation technique

Après la préparation de l'environnement, l'implémentation du Data Lake a été réalisée en plusieurs étapes afin de construire un pipeline complet de traitement des données industrielles.

3.1 Stockage des données dans MinIO

MinIO a été choisi comme solution de stockage objet compatible avec Amazon S3. Cette solution permet de reproduire une architecture Cloud tout en restant entièrement locale.

Quatre buckets ont été créés afin de représenter les différentes étapes du cycle de vie des données :

- **raw** : stockage des fichiers CSV d'origine provenant des lignes de production ;
- **staging** : stockage des données après harmonisation des schémas ;
- **curated** : stockage des jeux de données prêts pour l'analyse ;
- **archive** : stockage des anciennes versions des fichiers.

Cette organisation permet de séparer clairement les données brutes des données transformées et facilite leur maintenance.

3.2 Simulation des données industrielles

Le projet utilise des fichiers CSV représentant cinq lignes de production :

- Production Line A
- Production Line B
- Production Line C
- Production Line D
- Production Line E

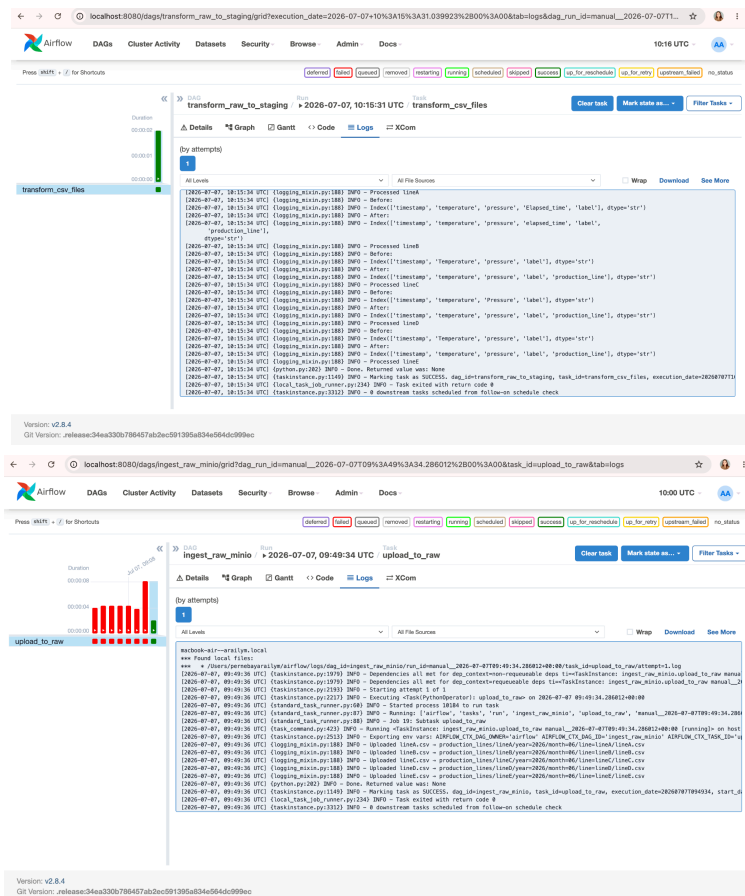
Chaque ligne produit des fichiers présentant des différences de structure (ordre des colonnes, noms des colonnes, valeurs manquantes, etc.). Cette situation reproduit un environnement industriel réel où plusieurs équipements génèrent des données hétérogènes.

3.3 Orchestration avec Apache Airflow

Apache Airflow est utilisé pour automatiser les différentes étapes du pipeline.

Un DAG d'ingestion a été développé afin d'exécuter automatiquement les traitements dans le bon ordre.

Figure 3: Interface Apache Airflow montrant l'exécution réussie du DAG_1 et DAG_2



Les principales tâches réalisées par ce pipeline sont :

- lecture des fichiers CSV ;
- calcul du hash MD5 afin de vérifier l'intégrité des fichiers ;
- copie des données vers le bucket **Raw** ;
- harmonisation du schéma des différents fichiers ;
- création des fichiers normalisés dans le bucket **Staging** ;
- génération des jeux de données finaux dans le bucket **Curated**.

Cette automatisation garantit la reproductibilité du traitement et limite les interventions manuelles.

3.4 Contrôle de l'intégrité des données

Avant chaque traitement, un contrôle d'intégrité est effectué.

Pour cela, une empreinte **MD5** est calculée pour chaque fichier.

Cette vérification permet de détecter une éventuelle modification ou corruption des données lors de leur transfert.

Même si cette méthode ne constitue pas un mécanisme de sécurité cryptographique, elle est largement utilisée pour contrôler l'intégrité des fichiers dans les pipelines de données.

3.5 Harmonisation des schémas

Les fichiers provenant des différentes lignes de production ne possèdent pas exactement la même structure.

Une étape de transformation a donc été développée afin de produire un schéma unique.

Cette harmonisation comprend notamment :

- renommage des colonnes ;
- réorganisation de leur ordre ;
- ajout des colonnes manquantes lorsque cela est nécessaire ;
- génération d'un format commun pour l'ensemble des fichiers.

Cette standardisation facilite les traitements analytiques réalisés sur les données.

À la fin de cette étape, le Data Lake est capable d'ingérer automatiquement les fichiers industriels, de vérifier leur intégrité, de les transformer puis de les stocker dans les différentes couches de MinIO selon leur niveau de traitement.

4. Intégration du catalogue de métadonnées OpenMetadata

Afin d'améliorer la gouvernance des données, OpenMetadata a été intégré au projet comme catalogue de métadonnées.

Son rôle est de centraliser les informations relatives aux jeux de données, aux services de stockage et aux pipelines afin de faciliter leur découverte et leur documentation.

4.1 Connexion entre MinIO et OpenMetadata

La première étape a consisté à créer un service de stockage S3 dans OpenMetadata.

La connexion a été configurée avec les paramètres suivants :

- endpoint MinIO local ;
- identifiants d'accès (**minioadmin**) ;
- région AWS (**us-east-1**) ;
- buckets du Data Lake.

Une première tentative de connexion a échoué car OpenMetadata ne pouvait pas communiquer avec MinIO. Le problème provenait du fait que les deux conteneurs Docker appartenaient à des réseaux différents.

Après avoir placé les conteneurs dans le même réseau Docker et configuré correctement l'endpoint MinIO, la connexion a été validée avec succès.

4.2 Découverte automatique des buckets

Une fois la connexion établie, OpenMetadata a détecté automatiquement les quatre buckets du Data Lake :

- **raw**
- **staging**
- **curated**
- **archive**

Des descriptions ainsi qu'un niveau de criticité (*Tier*) ont ensuite été renseignés afin de documenter les principaux espaces de stockage.

Cette étape améliore la compréhension de l'organisation des données au sein du Data Lake.

4.3 Difficultés rencontrées

L'objectif initial était de visualiser l'ensemble des fichiers CSV des lignes de production directement dans OpenMetadata.

Cependant, les buckets étaient visibles mais leur contenu n'était pas automatiquement exploré.

Après plusieurs recherches et essais, il est apparu que la découverte des objets présents dans un stockage S3 nécessite l'exécution d'un **workflow d'ingestion**.

Le pipeline d'ingestion OpenMetadata a donc été exécuté via Airflow. Après son exécution, les métadonnées du service de stockage ont été correctement synchronisées.

En revanche, OpenMetadata ne génère pas automatiquement le schéma des fichiers CSV présents dans MinIO. Cette fonctionnalité nécessite des workflows d'analyse supplémentaires qui n'entraient pas dans le périmètre du projet.

4.4 Limitations rencontrées

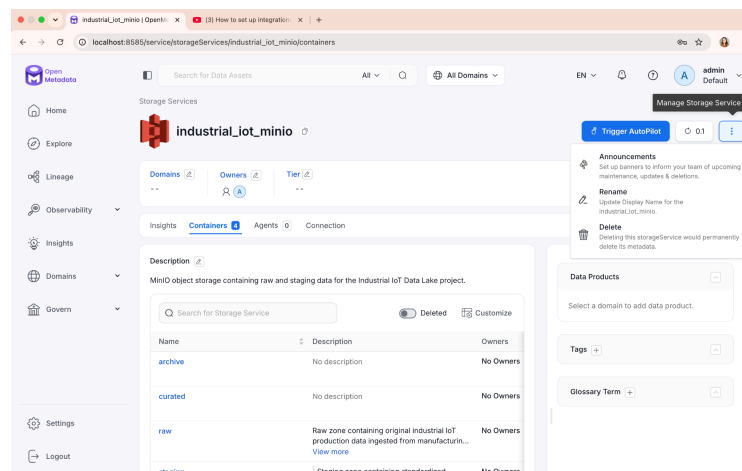
Deux limitations principales ont été identifiées durant le projet :

- la version locale de MinIO utilisée ne permettait pas de configurer les règles de cycle de vie (Lifecycle Management) depuis l'interface Web ;

- OpenMetadata référence les espaces de stockage et leurs métadonnées, mais n'analyse pas automatiquement le contenu des fichiers CSV sans configuration d'un workflow spécifique.

Ces limitations ont été validées avec l'encadrant et n'empêchent pas le bon fonctionnement de l'architecture mise en place.

Figure 3 : Interface OpenMetadata présentant le service de stockage `industrial_iot_minio` et les buckets détectés.



5. Conclusion

Ce projet a permis de concevoir un Data Lake industriel fonctionnel en reproduisant les principales étapes d'une architecture moderne de gestion des données.

L'environnement a été entièrement déployé à l'aide de Docker afin de garantir une installation reproductible et isolée. Les données industrielles simulées sous forme de fichiers CSV ont été stockées dans MinIO selon une organisation en plusieurs couches (*raw*, *staging*, *curated* et *archive*), reproduisant ainsi l'architecture d'un Data Lake utilisée en entreprise.

L'automatisation des traitements a été réalisée avec Apache Airflow. Le pipeline développé permet d'ingérer les fichiers, de vérifier leur intégrité grâce à un contrôle MD5, d'harmoniser les schémas provenant de différentes lignes de production puis de produire des jeux de données standardisés.

L'intégration d'OpenMetadata a permis de cataloguer le service de stockage MinIO, de documenter les différents buckets et d'améliorer la gouvernance des données. Les principales difficultés rencontrées concernaient la configuration de la connexion entre les conteneurs Docker, ainsi que les limites de l'exploration automatique des fichiers CSV par OpenMetadata. Ces problèmes ont été analysés puis résolus ou documentés lorsque la limitation provenait des outils eux-mêmes.

Ce projet m'a permis d'approfondir plusieurs compétences essentielles en ingénierie des données :

- déploiement d'une architecture distribuée avec Docker ;
- stockage objet avec MinIO ;
- orchestration de traitements avec Apache Airflow ;
- automatisation de pipelines de données en Python ;
- gouvernance des données avec OpenMetadata ;

- organisation des données selon une architecture Data Lake.

Annexes

- **Annexe A** : Architecture générale du projet (diagramme Draw.io)
- **Annexe B** : Interface MinIO présentant les buckets (*raw, staging, curated, archive*)
- **Annexe C** : Interface Apache Airflow montrant l'exécution réussie du DAG
- **Annexe D** : Interface OpenMetadata présentant le service `industrial_iot_minio` et les buckets détectés

Références

1. Apache Airflow <https://airflow.apache.org/docs/>
2. MinIO <https://hub.docker.com/r/minio/minio>
3. OpenMetadata <https://docs.open-metadata.org/v1.13.x/connectors/pipeline/airflow>
4. Docker
https://www.hostinger.com/tutorials/docker-tutorial/?utm_source=google&utm_medium=cpc&utm_id=18702076462&utm_campaign=Generic-Tutorials-DSA-t1|NT:Se|Lang:EN|LO:TR&utm_term=&utm_content=781713127657&gad_source=1&gad_campaignid=18702076462&gbraid=0AAAAADMy-hYCZ34PpMwI6bXkQDW_0uE6H&gclid=CjwKCAjw08fSBhA7EiwAfbQTsFv84wfWMqkCNgqb-gkzaRr8nqrNh9wIUqnHJ_8WnqQB9Tr7TwfAhoCscUQAvD_BwE
5. Amazon Web Services
https://docs.aws.amazon.com/s3/?icmpid=docs_homepage_featuredsvcs